

DA5300: Data Structures For Data Science

Laboratory 6 -Binary Search Trees and AVL Trees

Objectives

Implement Binary Search Trees (BSTs) and AVL trees. For both structures, implement insertion, deletion, inorder, preorder and postorder traversal, and display the tree in a text-file representation.

Exercise 1: Binary Search Tree

Create a node containing a key, a left-child reference and a right-child reference. Implement:

1. insertion;
2. deletion, including leaf, one-child and two-child cases;
3. inorder, preorder and postorder traversals;
4. display of the tree in a tree-like form.

Input

The input is a text file. Odd-numbered lines contain keys to be inserted, and even-numbered lines contain keys to be deleted. Keys on a line are comma-separated. Operations are cumulative.

```
50,30,70,20,40,60,80
30,80
10,35,65
50
```

Output

Produce four files: `inorder.txt`, `preorder.txt`, `postorder.txt`, and `tree.txt`. The three traversal files contain comma-separated keys, with exactly one line for every input line. Thus line i describes the tree after processing input line i .

Tree Display

`tree.txt` contains one tree for every input line. Consecutive trees are separated by a blank line. Each tree is displayed level by level. The last level starts at column 1 and its nodes are separated by exactly one space. Upper levels must use whitespace intelligently so that the displayed nodes represent the actual parent-child structure. Missing children must not be replaced by dummy keys. For example:

```
      50
     30  70
    20 40 60 80
```

The evaluator ignores upper-level positioning whitespace but checks the nodes and their levels, so different sensible indentation is permitted.

Exercise 2: AVL Tree

Implement an AVL tree with insertion, deletion, inorder, preorder and postorder traversal, and tree display. Store sufficient height information in each node to maintain

$$|h_L - h_R| \leq 1$$

at every node. Implement all four rotations: LL, RR, LR and RL. Deletion must also rebalance the tree.

Input and Output

Use exactly the same input and output format as Exercise 1. Odd lines insert all listed keys; even lines delete all listed keys. The operations are cumulative, and every output file has exactly as many lines as the input.

Program Execution

Your BST program must be named `bst.py`. Run:

```
python bst.py input.txt inorder.txt preorder.txt postorder.txt tree.txt
```

Your AVL program must be named `avl.py`. Run:

```
python avl.py input.txt inorder.txt preorder.txt postorder.txt tree.txt
```

Do not hard-code file names.

Test Cases

The following three inputs are to be used for both exercises.

Test 1

```
50,30,70,20,40,60,80
30,80
10,35,65
50
```

Test 2

```
40,20,60,10,30,50,70,25
10
30
60
```

Test 3

```
30,20,10,25,28,27,50,60
10,60
15,18,17
25,50
```

Evaluation

An evaluator `evaluate.py` independently constructs the required BST or AVL tree and checks all three traversals, the number of output lines, and the tree structure. Run it as:

```
python evaluate.py bst test1.txt inorder.txt preorder.txt postorder.txt
tree.txt
python evaluate.py avl test1.txt inorder.txt preorder.txt postorder.txt
tree.txt
```

Repeat with `test2.txt` and `test3.txt`. It prints PASS or FAIL and identifies the first failed component.

Submission

Submit `bst.py`, `avl.py`, and the required output files for all three test cases. Programs must work for arbitrary valid input, not only the supplied tests.